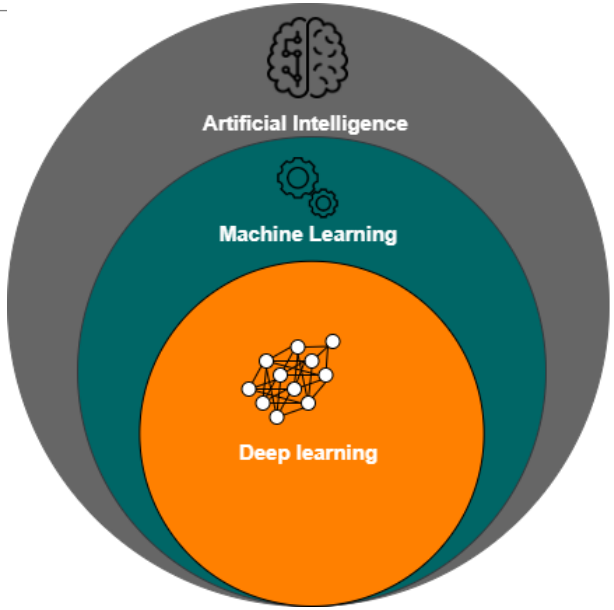
INTRODUCTION TO MACHINE LEARNING

Fundamentals of Machine Learning

Machine learning Overview

- ***What is Machine learning?***
  - *Machine Learning combines computer science, mathematics, and statistics*
  - *The most popular technique of predicting the future or classifying information to help people in making necessary decisions*
  - *Machine Learning algorithms are trained over instances or examples through which they learn from past experiences.*

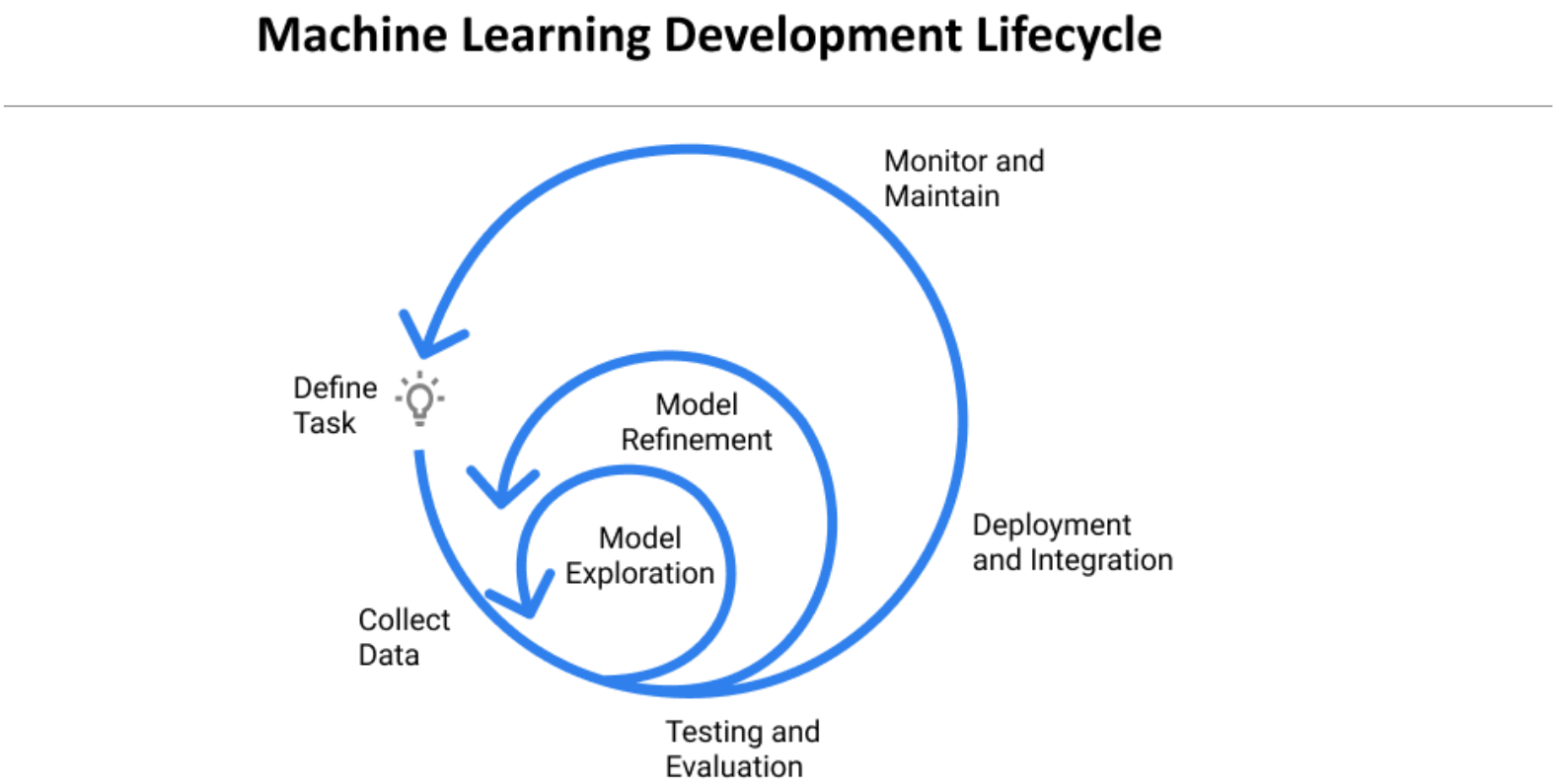


Machine learning Overview

Why Is Machine Learning Important?

- *ML can help make decisions faster from data*
- *Computational processing that is cheaper and more powerful*

Machine learning Overview



Machine learning Overview

Supervised Learning

- Regression, Decision Tree, SVM, …

Unsupervised Learning

- K-Mean, DBSCAN, …

Reinforcement Learning

Machine learning Overview

Application of machine learning

- Recommendation Systems
- Customer Segmentation
- Stock Market trading
- Object detection
- Online Fraud Detection
- ….

REGRESSION

Fundamentals of Machine Learning

Agenda

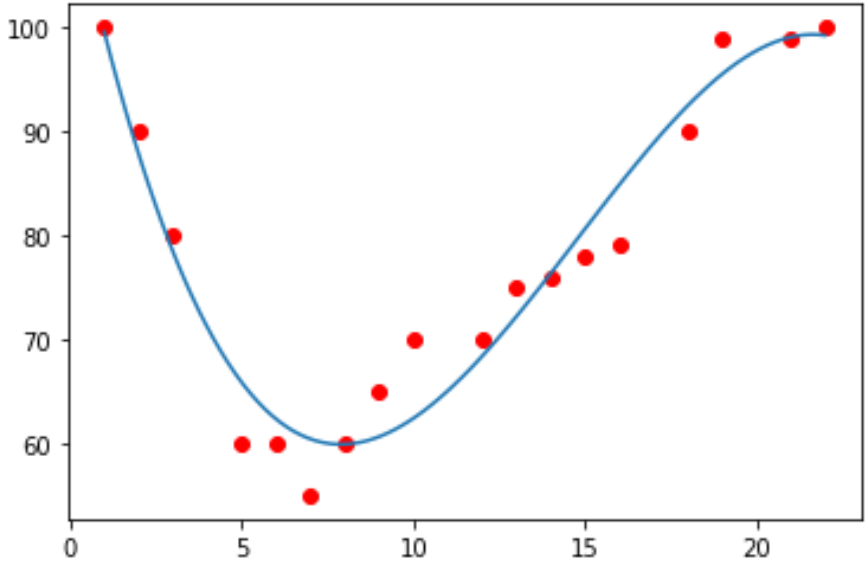
Regression Overview

Linear Regression

Multiple Regression

Regression Overview

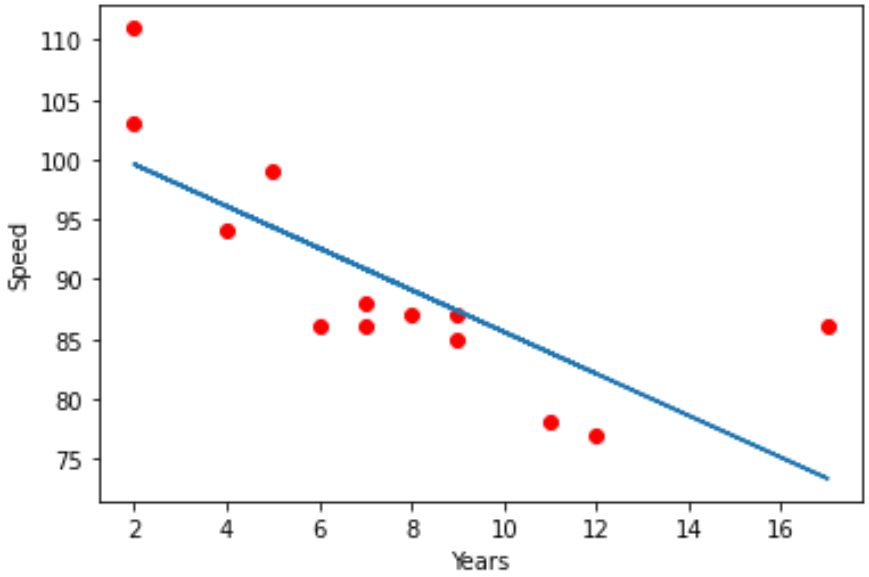
- ***Try to find the relationship between variables.***
- ***Regression is the process of predicting continuous values.***
- ***Linear regression uses the relationship between the data-points to draw a straight line through all them.***
- ***Application:***
  - ✓ *Trend line*
  - ✓ *Finance*
  - ✓ *…*



Regression Overview

Types of the regression models

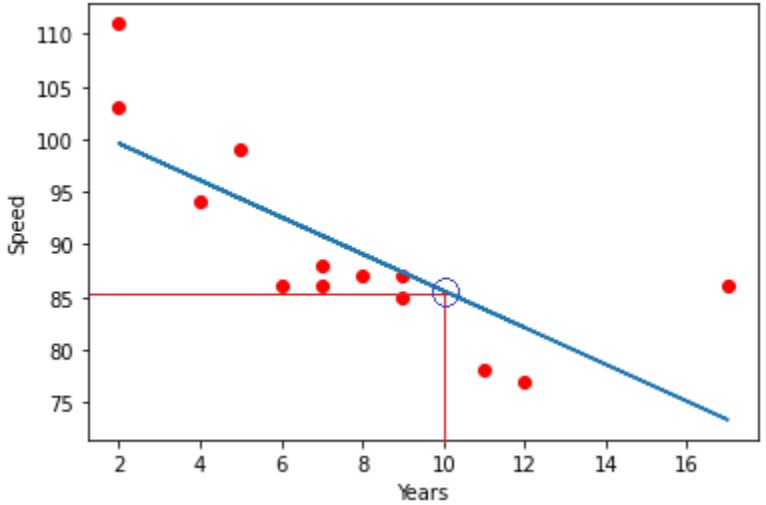
- Simple Regression
  - Simple linear Regression
  - Simple Non-linear Regression
- Multiple Regression
  - Multiple Linear Regression
  - Multiple Non-linear Regression



Linear Regression

How does linear regression work?

Years	Speed
5	99
7	86
8	87
7	88
2	111
17	86
2	103
9	87
4	94
10	?

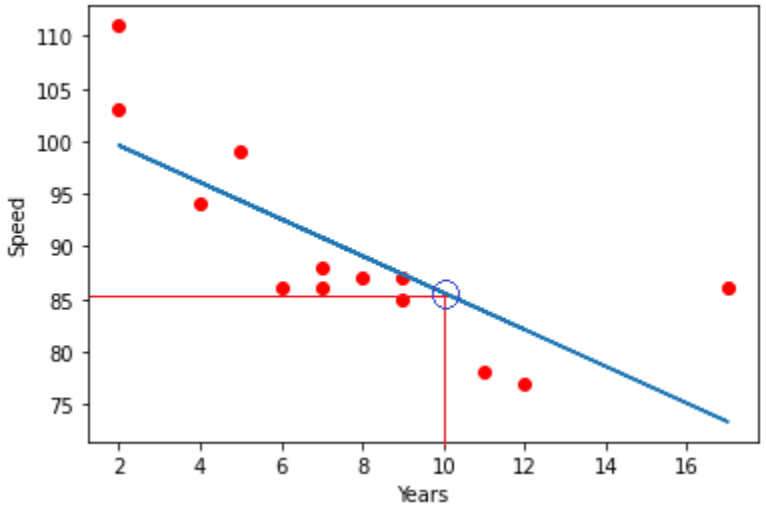


Linear Regression

How does linear regression work?

$$\hat{Y} = \theta_0 + \theta_1 X$$

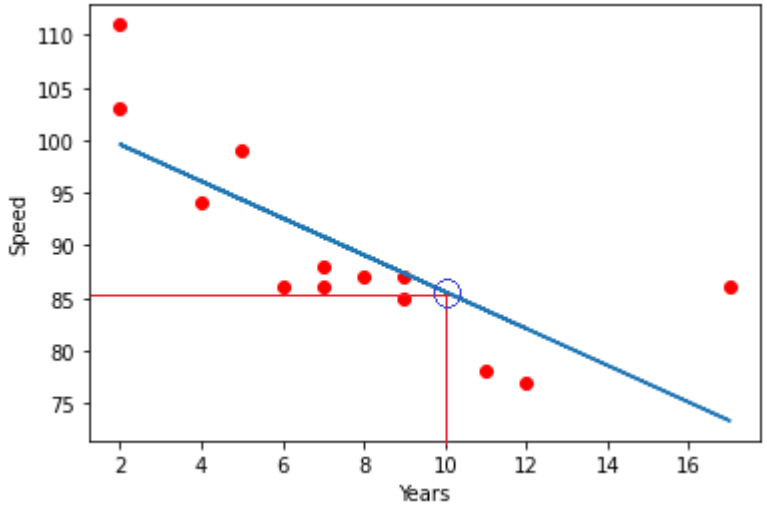
$\hat{Y}$: *Predict*
θ_0: *intercept*
θ_1: *Slope or Coefficient*



Linear Regression

How does linear regression work?

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.linear_model import LinearRegression
x = [[5],[7],[8],[7],[2],[17],[2],[9],[4],[11],[12],[9]
,[6]]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]
model = LinearRegression()
model.fit(x, y)
coef = model.coef_
intercept = model.intercept_
def myfunc(x):
  return coef * x + intercept
mymodel = list(map(myfunc, x))
plt.scatter(x, y, color='red')
plt.xlabel('Years')
plt.ylabel('Speed')
plt.plot(x, mymodel)
plt.show
```



Linear Regression

How to find the best fit?

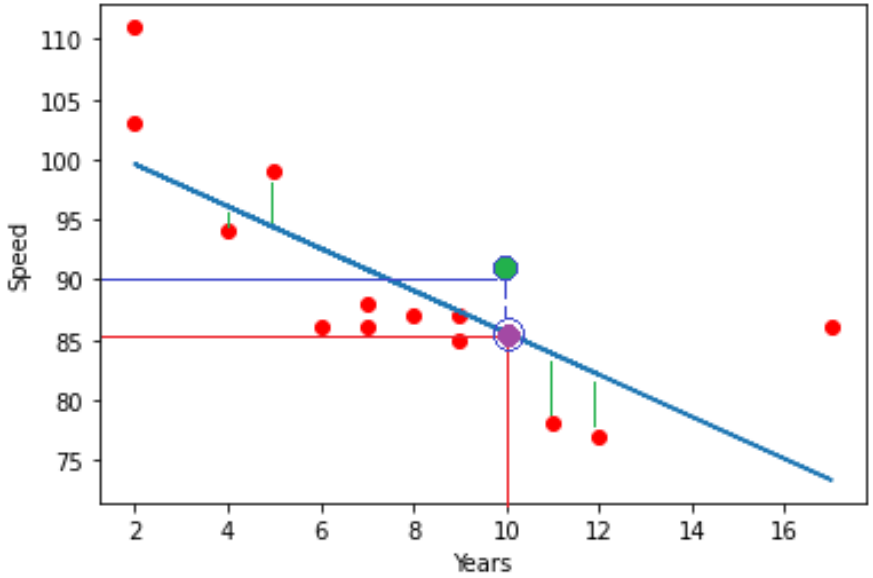
$x_1 = 10$ *independent variable*

$y = 90$ *actual speed of* x_1

$\hat{y} = \theta_0 + \theta_1 x$

$\hat{y} = 85.59$ *the predicted speed of* x_1

$Error = y - \hat{y} = 90 - 85.59 = 4.41$



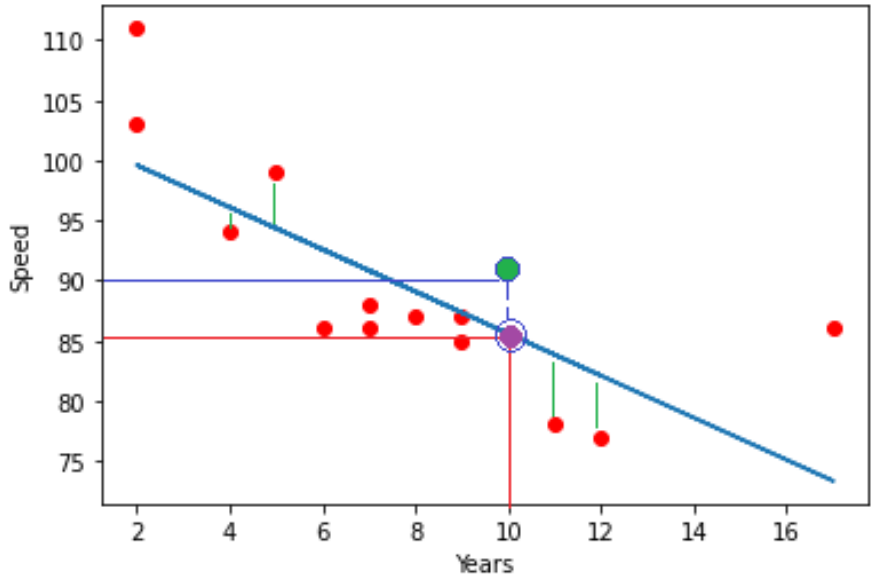
Linear Regression

How to find the best fit?

$Error = y - \hat{y} = 90 - 85.59 = 4.41$

Mean Square Error

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$



Linear Regression

How to find the best fit?

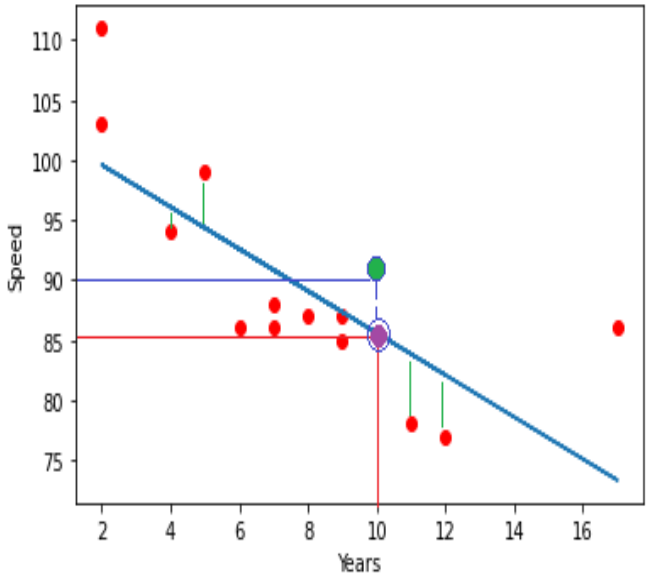
$$Mean\ Square\ Error\ -\ MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \widehat{y_i})^2$$

$$Mean\ Absolute\ Error\ -\ MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \widehat{y_i}|$$

$$Root\ Absolute\ Error\ -RAE = \frac{\sum_{i=1}^{n}|y_i - \widehat{y_i}|}{\sum_{i=1}^{n}|y_i - \bar{y}|}$$

$$Root\ Square\ Error\ -RSE = \frac{\sum_{i=1}^{n}(y_i - \widehat{y_i})^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

$$R^2 = 1 - RSE$$



Linear Regression

Estimating the parameters: Least Squares Estimation

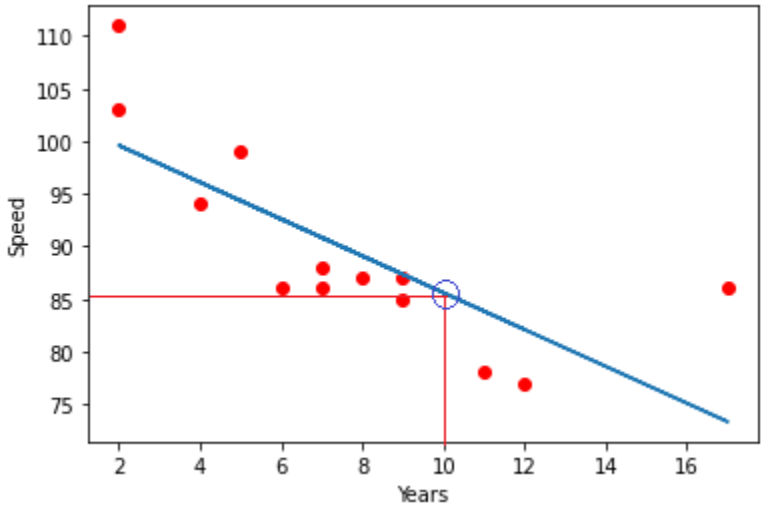
$\hat{y} = \theta_0 + \theta_1 x$

$\hat{Y}$: *Predict*
θ_0: *intercept*
θ_1: *Slope or Coefficient*

$$\theta_1 = \frac{\sum_i^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_i^n (x_i - \bar{x})^2}$$

$$\theta_0 = \bar{y} - \theta_1 \bar{x}$$

$\bar{y}$: *mean of* y
$\bar{x}$: *mean of* x



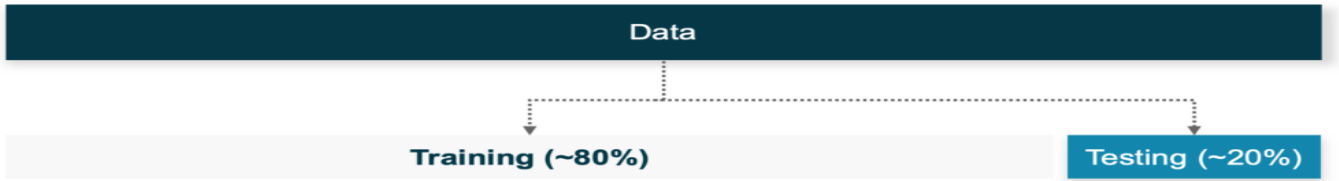
Linear Regression

Train/Test split evaluation approach

Test on a portion of train set

- Split dataset: 80% to training and 20% to testing
- Using sklearn for split dataset.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, label, test_size=0.2)
# test_size should be between 0.0 and 1.0, shuffle default = True
```

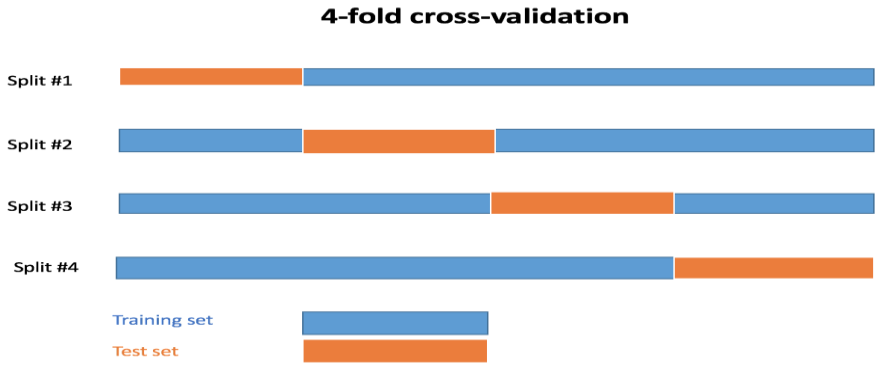


Linear Regression

Train/Test split evaluation approach

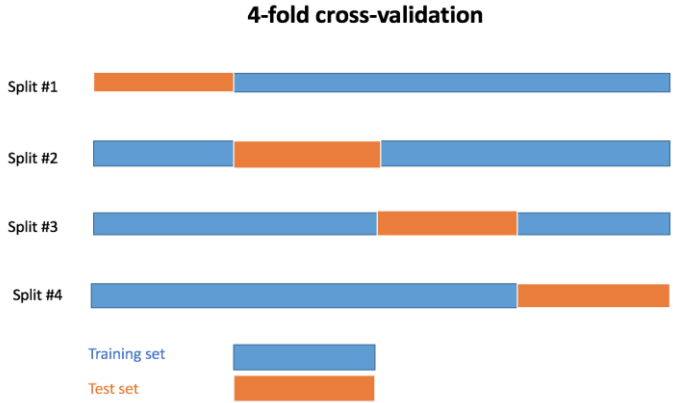
Cross-validation

- Used to evaluate machine learning models on a limited data sample.



Linear Regression

Train/Test split evaluation approach



Cross-validation

- Using cross-validation with sklearn

```
from sklearn.model_selection import KFold
kf = KFold(n_splits=4)
X = # independent variable
y = # dependent variable
for train_index, test_index in kf.split(X)
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
```

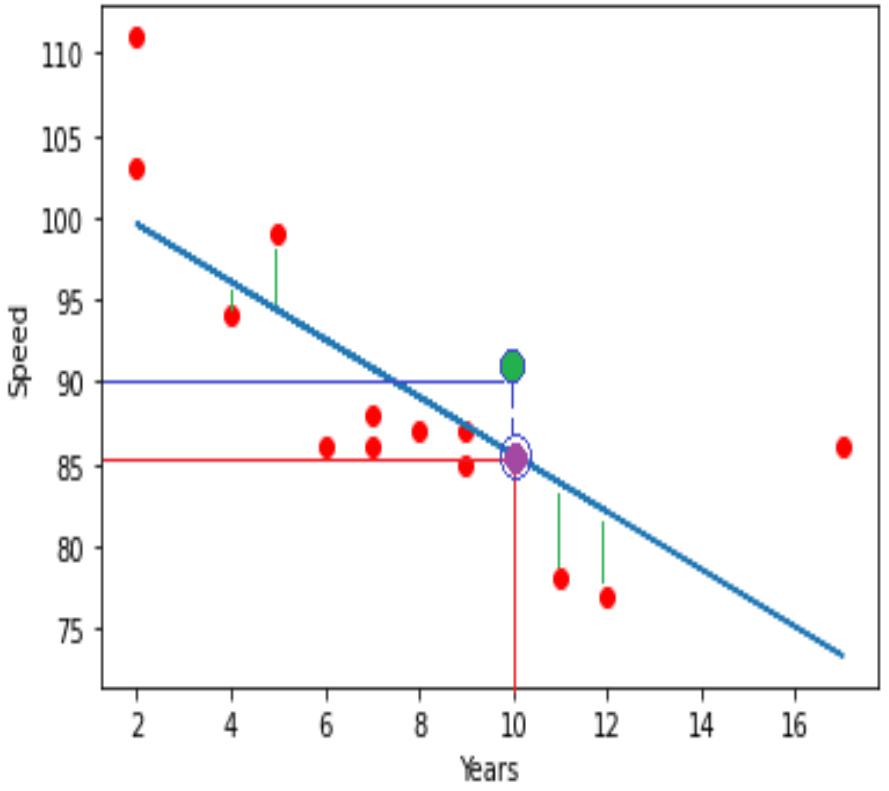
Linear Regression

Advantage of linear regression

Very fast

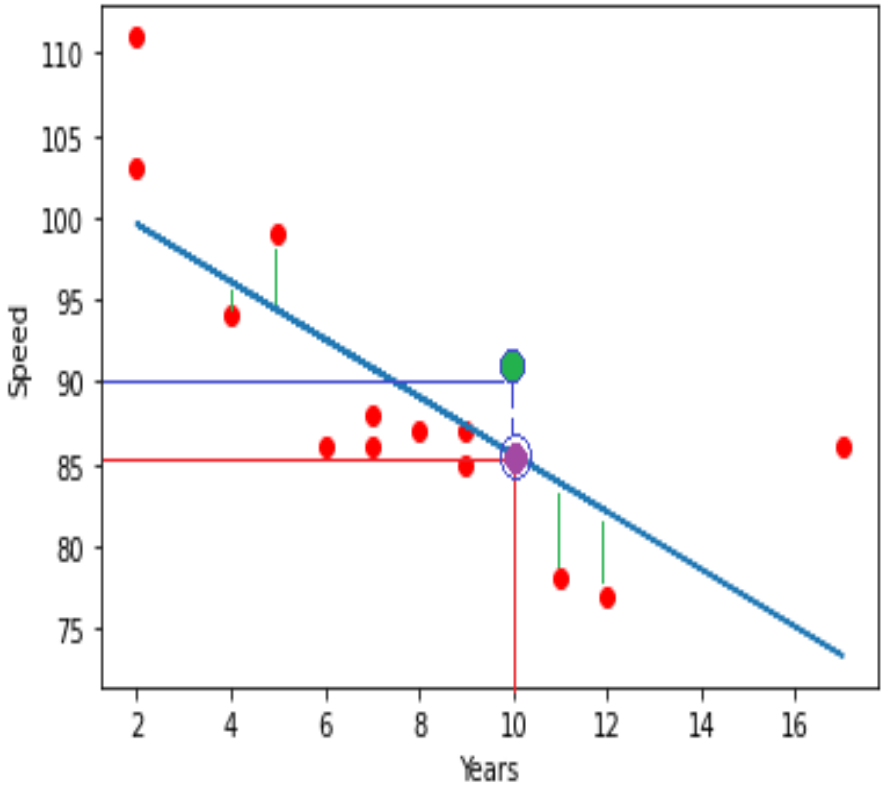
No parameter tuning

Easy to understand, and highly interpretable



Linear Regression

Practice



Multiple Regression

Like Linear Regression

Try to predict a value based on two or more variables

ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
2.4	4	9.2	212
2.4	4	9.8	225
3.5	6	10.4	239
5.9	12	15.6	359
5.9	12	15.6	359
4.7	8	14.7	338
4.7	8	15.4	354
4.7	8	14.7	338
4.7	8	15.4	354
5.9	12	15.6	?

X: Independent variable

Y: Dependent variable

Multiple Regression

Predicting continuous values with multiple linear regression

$$Co2Emissons = \theta_0 + \theta_1 EngineSize + \theta_2 Cylinders + \cdots$$

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_n x_n$$

$$\hat{y} = \theta^T X$$

$$\theta^T = [\theta_0, \theta_1, \theta_2, ..] \qquad X = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_3 \\ \ldots \end{bmatrix}$$

ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
2.4	4	9.2	212
2.4	4	9.8	225
3.5	6	10.4	239
5.9	12	15.6	359
5.9	12	15.6	359
4.7	8	14.7	338
4.7	8	15.4	354
4.7	8	14.7	338
4.7	8	15.4	354
5.9	12	15.6	?

X: Independent variable

Y: Dependent variable

Multiple Regression

Using MSE to expose the errors in the model

$\hat{y} = \theta^T X$

$\hat{y} = 200$ The predicted emission of x_i

$y = 212$ Actual value of x_i

$y_i - \hat{y_i} = 212 - 200 = 12$ Residual error

ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
2.4	4	9.2	212
2.4	4	9.8	225
3.5	6	10.4	239
5.9	12	15.6	359
5.9	12	15.6	359
4.7	8	14.7	338
4.7	8	15.4	354
4.7	8	14.7	338
4.7	8	15.4	354
5.9	12	15.6	?

X: Independent variable

Y: Dependent variable

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y_i})^2$$

➡ The objective of multiple linear regression is to minimize the MSE equation

Multiple Regression

How to estimate θ?

Least Squares

- Linear algebra operations
- Takes a long time for large dataset (10K+ rows)

An optimization algorithm

- Gradient Descent
- Proper approach if you have a very large dataset

Multiple Regression

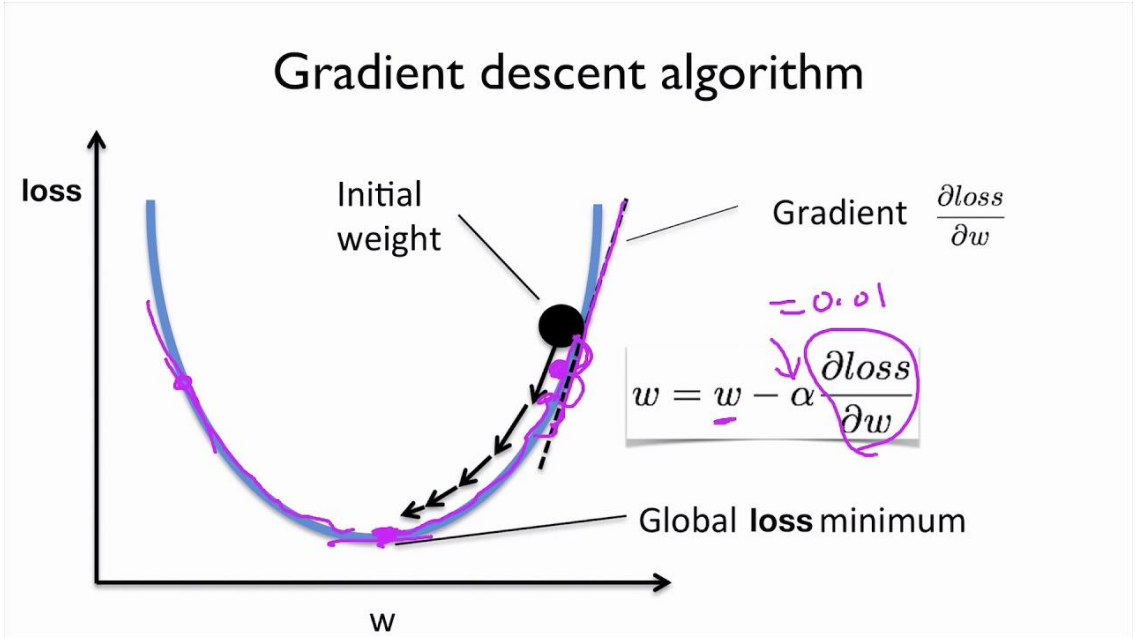
Gradient Descent

$\hat{y} = W^T X$

$$L(W) = MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$w_{i,t+1} = w_{i,t} - \propto \frac{\partial L(W)}{\partial w_{i,t}}$$

Where: $\propto$ *is learning rate*



Multiple Regression

Advantages of Multiple Regression

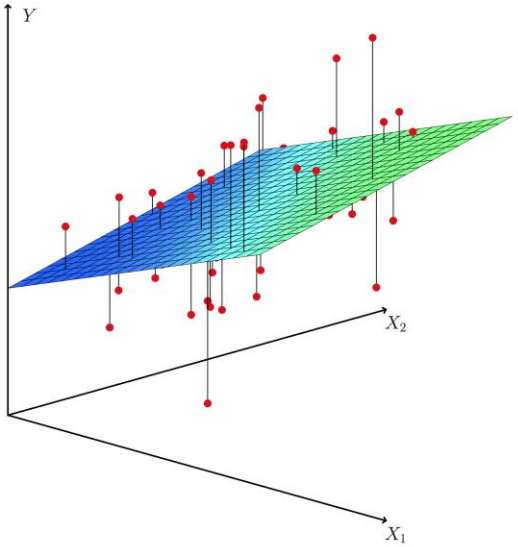
- The first is the ability to determine the relative influence of one or more predictor variables to the criterion value.
- The second advantage is the ability to identify outliers, or anomalies

Disadvantages of Multiple Regression

- Not good with incomplete data, possibly misleading

Multiple Regression

Practice



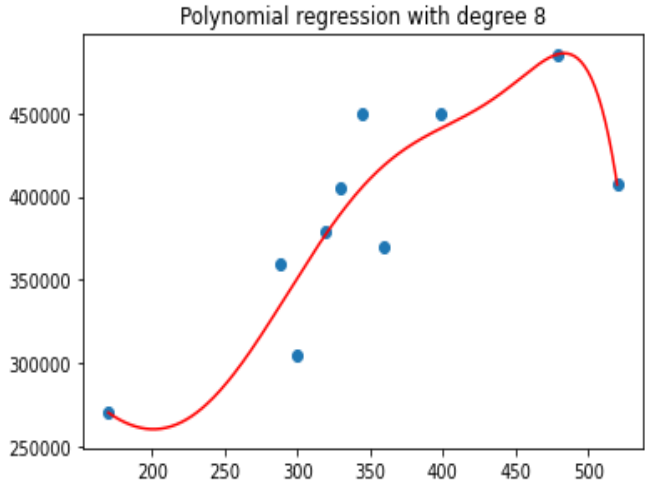
Regression

POLYNOMIAL REGRESSION

Polynomial Regression

Relationship between the independent variable x and dependent variable y is modeled as an ***nth*** degree polynomial

Polynomial regression fits a nonlinear relationship between the value of x and the corresponding conditional mean of y, denoted E(y |x)



Polynomial Regression

Why Polynomial Regression ?

There are some relationships that a researcher will hypothesize is curvilinear. Clearly, such type of cases will include a polynomial term.

Inspection of residuals. If we try to fit a linear model to curved data, a scatter plot of residuals (Y axis) on the predictor (X axis) will have patches of many positive residuals in the middle. Hence in such situation it is not appropriate.

An assumption in usual multiple linear regression analysis is that all the independent variables are independent. In polynomial regression model, this assumption is not satisfied.

Polynomial Regression

Simple linear Regression

- $y = \theta_0 + \theta_1 x$

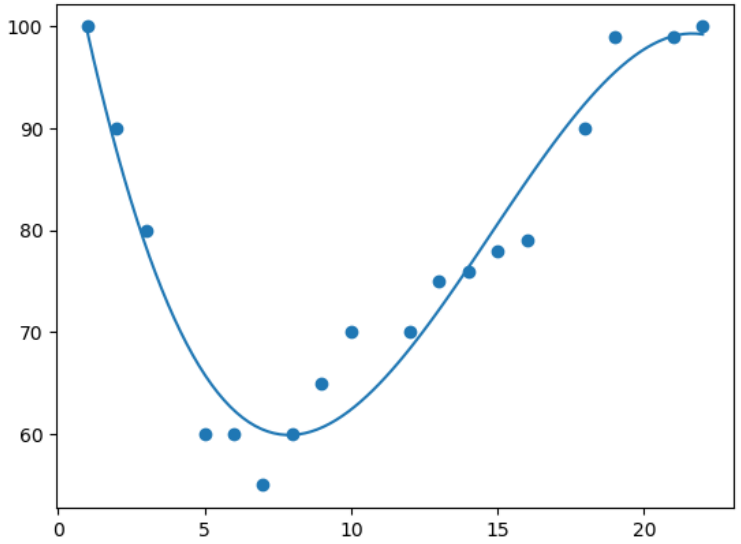
Polynomial Regression

- $y = \theta_0 + \theta_1 x + \theta_2 x^2 + \ldots + \theta_n x^n$

Polynomial Regression

Polynomial Regression

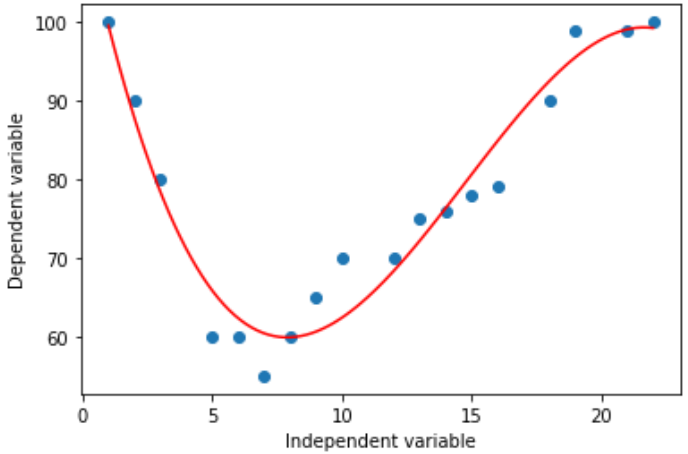
- $y = \theta_0 + \theta_1 x + \theta_2 x^2 + \ldots + \theta_n x^n$



Polynomial Regression

Simple Polynomial Regression ?

```
import numpy
import matplotlib.pyplot as plt

x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]

poly_model = numpy.poly1d(numpy.polyfit(x, y, 3))

myline = numpy.linspace(1, 22, 100)

plt.scatter(x, y)
plt.plot(myline, poly_model(myline), c='red')
plt.xlabel('Independent variable')
plt.ylabel('Dependent variable')
plt.show()
```



Polynomial Regression

R^2 Score

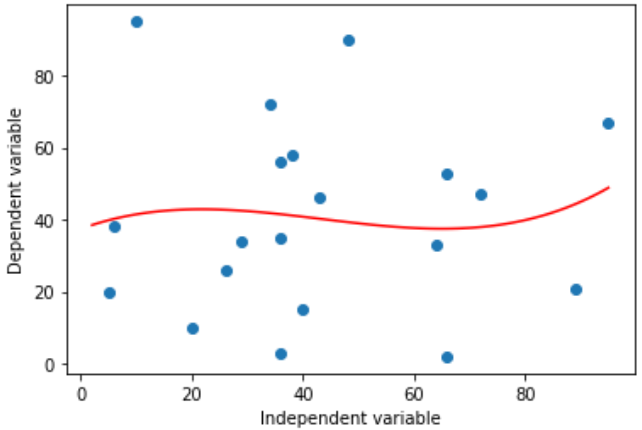
- The r-squared value ranges from 0 to 1 where 0 means no relationship, and 1 means 100% related.

```
import numpy
from sklearn.metrics import r2_score

x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]

mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))

print(r2_score(y, mymodel(x)))
#0.9432150416451025
```

Polynomial Regression

Bad Fit?

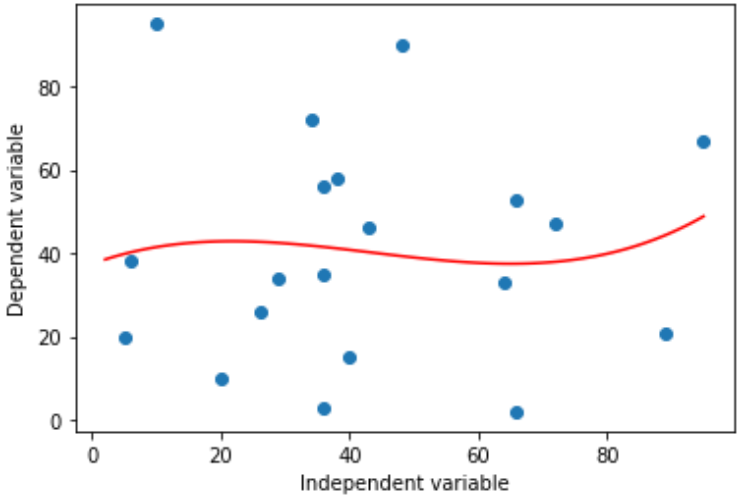
```
import numpy
import matplotlib.pyplot as plt

x = [89,43,36,36,95,10,66,34,38,20,26,29,48,64,6,5,36,66,72,40]
y = [21,46,3,35,67,95,53,72,58,10,26,34,90,33,38,20,56,2,47,15]

bad_model = numpy.poly1d(numpy.polyfit(x, y, 3))

myline = numpy.linspace(2, 95, 100)

plt.scatter(x, y)
plt.plot(myline, bad_model(myline), c='red')
plt.xlabel('Independent variable')
plt.ylabel('Dependent variable')
plt.show()
```



Polynomial Regression

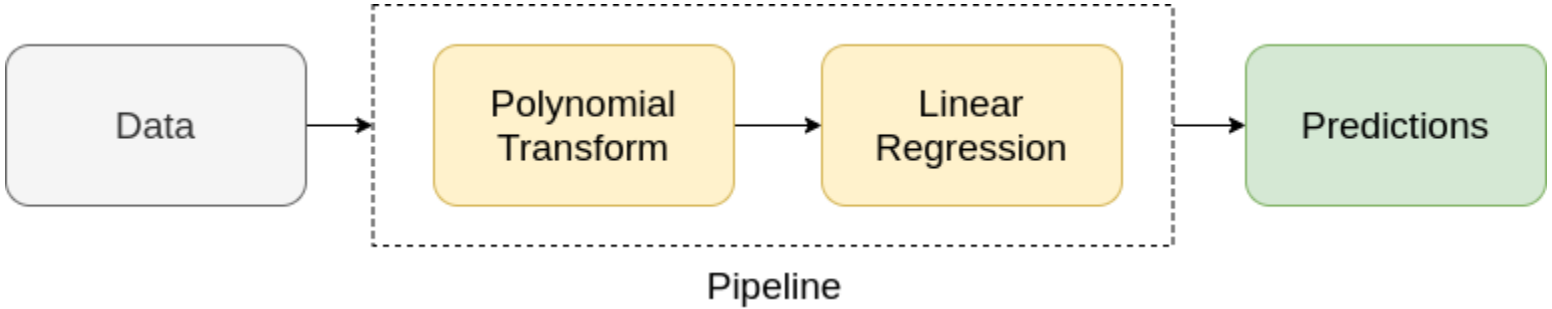
Bad Fit ? – R^2 Score

```
import numpy
from sklearn.metrics import r2_score

x = [89,43,36,36,95,10,66,34,38,20,26,29,48,64,
6,5,36,66,72,40]
y = [21,46,3,35,67,95,53,72,58,10,26,34,90,33,3
8,20,56,2,47,15]

bad_model = numpy.poly1d(numpy.polyfit(x, y, 3)
)

print(r2_score(y, bad_model(x)))
# 0.009952707566680652
```



Polynomial Regression

With sklearn



Polynomial Transform – *PolynomialFeatures*

- Generate polynomial and interaction features.

Linear Regression

- Like Multiple Linear Regression

Polynomial Regression

With sklearn

Polynomial Transform – *PolynomialFeatures*

- *Ex: Input sample is two dimensional and of the form [a, b], the degree-2 polynomial features are [1, a, b, a^2, ab, b^2].*

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
X = np.arange(6).reshape(3, 2)
print(X)
poly = PolynomialFeatures(degree=2)
poly.fit_transform(X)
```

Polynomial Regression

With sklearn

Creating pipeline and fitting it on data

```
# importing libraries for polynomial transform
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

poly_features = PolynomialFeatures(degree=2)
linear_model = LinearRegression()

x_transform = poly_features.fit_transform(x.reshape(-1, 1))
linear_model.fit(x_transform, y.reshape(-1,1))
```

Polynomial Regression

Uses of Polynomial Regression:

These are basically used to define or describe non-linear phenomenon such as:

- Growth rate of tissues.
- Progression of disease epidemics
- Distribution of carbon isotopes in lake sediments

Polynomial Regression

Advantages of using Polynomial Regression

Broad range of function can be fit under it.

Polynomial basically fits wide range of curvature.

Polynomial provides the best approximation of the relationship between dependent and independent variable.

Polynomial Regression

Disadvantages of using Polynomial Regression

These are too sensitive to the outliers.

The presence of one or two outliers in the data can seriously affect the results of a nonlinear analysis.

In addition there are unfortunately fewer model validation tools for the detection of outliers in nonlinear regression than there are for linear regression.

NON-LINEAR
REGRESSION

Regression

Non-linear Regression(NLR)

NLR is any relationship between an independent variable ***X*** and a dependent variable ***y*** which results in a non-linear function modelled data.

Essentially any relationship that is **not** linear, can be termed as non-linear and is usually represented by the polynomial of ***k*** degrees (maximum power of ***X)***.

Non-linear Regression(NLR)

There are many types of non-linear regression, but perhaps the most common are:

- Cubic
- Quadratic
- Exponential
- Logarithmic
- Sigmoidal / Logistic

Non-linear Regression(NLR)

Cubic

- $y = \theta_0 + 1x^3 + 1x^2 + 1x^1$

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(-5.0, 5.0, 0.1)

y = 1*(x**3) + 1*(x**2) + 1*x + 3
y_noise = 20 * np.random.normal(size=x.size)
ydata = y + y_noise
plt.plot(x, ydata,  'bo')
plt.plot(x,y, 'r')
plt.ylabel('Dependent Variable')
plt.xlabel('Indepdendent Variable')
plt.show()
```

Non-linear Regression(NLR)

Quadratic

- $y = x^2$

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(-5.0, 5.0, 0.1)

y = np.power(x,2)
y_noise = 2 * np.random.normal(size=x.size)
ydata = y + y_noise
plt.plot(x, ydata,  'bo')
plt.plot(x,y, 'r')
plt.ylabel('Dependent Variable')
plt.xlabel('Indepdendent Variable')
plt.show()
```

Non-linear Regression(NLR)

Exponential

- $y = a + bc^x$

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(-5.0, 5.0, 0.1)

Y= np.exp(X)

plt.plot(X,Y)
plt.ylabel('Dependent Variable')
plt.xlabel('Indepdendent Variable')
plt.show()
```

Non-linear Regression(NLR)

Logarithmic

- $y = \log x$

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(-5.0, 5.0, 0.1)

Y = np.log(X)

plt.plot(X,Y)
plt.ylabel('Dependent Variable')
plt.xlabel('Indepdendent Variable')
plt.show()
```

Non-linear Regression(NLR)

Sigmoidal/Logistic

- $y = a + \frac{b}{1+c^{(X-d)}}$

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(-5.0, 5.0, 0.1)

Y = 1-4/(1+np.power(3, X-2))

plt.plot(X,Y)
plt.ylabel('Dependent Variable')
plt.xlabel('Indepdendent Variable')
plt.show()
```

Non-linear Regression(NLR)

Building The Model

- Use **curve_fit** which uses non-linear least squares to fit
- Ex: $\hat{y} = \frac{1}{1+e^{\beta_1(X-\beta_2)}}$

```
def sigmoid(x, Beta_1, Beta_2):
     y = 1 / (1 + np.exp(-Beta_1*(x-Beta_2)))
     return y
```

```
from scipy.optimize import curve_fit
popt, pcov = curve_fit(sigmoid, xdata, ydata)
#print the final parameters
print(" beta_1 = %f, beta_2 = %f" % (popt[0], popt[1]))
```

Thank you

Linear Regression

Gradient Descent

$\hat{y} = W^T X$

$$L(W) = MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$w_{i,t+1} = w_{i,t} - \propto \frac{\partial L(W)}{\partial w_{i,t}}$$

Where: $\propto$ *is learning rate*